# ADVANCED SUPERVISED LEARNING PROJECTS

An end-to-end engineering guide

**PROJECT A | NASA C-MAPSS Remaining Useful Life Regression**

**PROJECT B | TOC-UCO Ordinal Classification**

*A practical build plan for two reproducible, leakage-aware ML systems—with target design, validation, error analysis, and deployment.*

Source review: 20 October 2026

## Table of contents

1. How to use this guide
2. The two-project strategy
3. Important dataset clarifications
4. Shared engineering workflow and repository layout
5. Project A — C-MAPSS RUL regression
- Problem, data, and target
- Phase A: data and feature pipeline
- Phase B: split, baseline, and model comparison
- Evaluation, interpretation, and deployment
6. Project B — TOC-UCO ordinal classification
- Problem, data, and ordinal labels
- Phase A: data and preprocessing
- Phase B: nominal versus ordinal models
- Evaluation, interpretation, and deployment
7. Reproducibility and experiment tracking
8. Six-week build roadmap
9. Portfolio README and interview narrative
10. Definition-of-done checklist
11. References

## 1. How to use this guide

Build the projects in the order shown. Do not start with LSTMs, Transformers, neural ordinal models, SHAP, or hyperparameter optimization. First establish a valid dataset pipeline and a baseline. Only add complexity when an experiment answers a specific question.

For every phase, keep four artifacts:

- **Code:** repeatable scripts, not only notebook cells.
- **Evidence:** metric tables, plots, split definitions, and error analysis.
- **Decision log:** what you chose, why you chose it, and what alternatives you tested.
- **Reproducibility record:** dataset version, random seed, package versions, configuration, and run ID.

A project is not complete merely because it achieves a good score. It is complete when another developer can reproduce the score, understand the evaluation, send valid inputs to the model, and identify the model's limitations.

### What these projects are meant to teach

| Skill | C-MAPSS RUL regression | TOC-UCO ordinal classification |
|---|---|---|
| Problem type | Predict remaining cycles until failure | Predict an ordered class |
| Main trap | Leakage across time and engines | Treating ordered labels as unrelated categories |
| Main design choice | RUL target, trailing features, engine-level validation | Ordinal loss or threshold model, ordered evaluation |
| Primary metrics | RMSE, MAE, NASA asymmetric score | MAE on class index, Quadratic Weighted Kappa (QWK), off-by-one accuracy |
| Deployment output | Non-negative RUL estimate | Ordered class, class probabilities, cumulative probabilities |

## 2. The two-project strategy

### Project A in one sentence

Use past operating settings and sensor readings from a simulated turbofan engine to estimate how many operational cycles remain before failure.

### Project B in one sentence

Use tabular measurements to predict an ordered outcome while explicitly measuring how far a wrong prediction is from the true class.

Together, the projects demonstrate two different forms of modeling judgment: respecting the data's **time structure** and respecting the target's **ordinal structure**.

### Your final portfolio should show

12. A meaningful baseline for each problem.
13. A split policy that matches the data-generation process.
14. A clear feature and target definition.
15. A comparison between a simple model and at least one stronger model.
16. Metrics suited to the actual task, including error analysis.
17. A saved inference pipeline and a small API or demo.
18. A README that explains decisions and limitations rather than only presenting the best score.

## 3. Important dataset clarifications

These details correct or qualify points in the original research notes. Dataset versions and repository metadata matter; do not silently mix differently processed versions.

### C-MAPSS

The NASA dataset description states that each row has 26 numeric columns: engine/unit ID, cycle, three operational settings, and 21 sensor measurements. It has four subsets: FD001, FD002, FD003, and FD004. The Zenodo record linked below currently lists a downloadable `CMAPSSData.zip`; use it as a practical source and cite the NASA dataset page for the dataset description. [1][2]

### TOC-UCO

The official TOC-UCO repository describes 46 datasets and 30 randomized train/test partitions. Its currently listed `abalone` and `bank1` datasets each have **nine classes**, not five. The five-class names “Abalone5” and “Bank5” in the original notes therefore should not be assumed to match the official benchmark table. For a direct TOC-UCO reproduction, use the dataset names and class counts in the official repository. If you use a separately binned five-class variant, document its precise source and binning procedure and do not present it as the same official configuration. [3][4]

**Recommended start:** use `abalone` from the official TOC-UCO package, preserving its official class order and split files. If you specifically want a five-class exercise, select a five-class dataset listed by the repository, such as `winequalityRed`, and document the exact benchmark definition.

## 4. Shared engineering workflow

Use the same high-level pipeline for both projects:

**Problem definition → data contract → split policy → baseline → preprocessing and features → model training → validation → error analysis → final selection → test evaluation → serialization → deployment → monitoring plan.**

### Recommended repository structure

```
advanced-ml-projects/
├── README.md
├── requirements.txt
├── .gitignore
├── configs/
│   ├── rul_fd001.yaml
│   └── ordinal_abalone.yaml
├── data/
│   ├── raw/                # not committed if large or license-sensitive
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 00_rul_data_audit.ipynb
│   ├── 01_rul_feature_checks.ipynb
│   ├── 02_ordinal_data_audit.ipynb
│   └── 03_ordinal_error_analysis.ipynb
├── src/
│   ├── common/
│   │   ├── config.py
│   │   └── metrics.py
│   ├── rul/
│   │   ├── data.py
│   │   ├── features.py
│   │   ├── train.py
│   │   ├── evaluate.py
│   │   └── api.py
│   └── ordinal/
│       ├── data.py
│       ├── preprocessing.py
│       ├── train.py
│       ├── evaluate.py
│       └── api.py
├── tests/
│   ├── test_rul_features.py
│   ├── test_rul_contract.py
│   └── test_ordinal_probabilities.py
├── artifacts/              # saved pipelines; usually not all committed
└── reports/
    ├── figures/
    ├── metrics.csv
    └── decision_log.md
```

### Environment

Start with Python, NumPy, pandas, scikit-learn, Matplotlib, Seaborn, Jupyter, joblib, FastAPI, Uvicorn, and pytest. Add Optuna and SHAP only after the baseline and evaluation loop are stable. Add XGBoost or LightGBM only when you are ready to compare a boosted-tree model.

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install numpy pandas scikit-learn matplotlib seaborn jupyter joblib fastapi "uvicorn[standard]" pytest
```

After the environment works, save the exact environment used for a reported result:

```
pip freeze > requirements-lock.txt
```

Use `requirements.txt` for intentional direct dependencies and a lock/export file for the environment that produced the published results. Do not claim a model is reproducible merely because the package names are listed.

### Non-negotiable leakage rule

Any fitted transformation—imputation, scaling, feature selection, clustering, probability calibration, or threshold adjustment—must learn its parameters from the training portion of the current split only. Apply the fitted transformation unchanged to validation and test data. A scikit-learn `Pipeline` or `ColumnTransformer` helps enforce this rule.

# 5. Project A — NASA C-MAPSS Remaining Useful Life Regression

## 5.1 Problem and data contract

Each engine has a trajectory of operating cycles. Each row contains an engine ID, cycle number, three operating settings, and 21 sensor readings. Training engines run to simulated failure. Test trajectories stop before failure, and a separate file supplies the true remaining useful life at the final observed point. [1][2]

**Prediction contract:** given the permitted sensor history for one engine up to cycle `t`, produce an estimate of the number of operational cycles remaining after `t`.

**Start with FD001.** It has one operating condition and one fault mode, so it is a manageable first benchmark. Move to FD002 after the pipeline is correct; its multiple operating conditions make preprocessing harder. FD003 and FD004 can be later robustness extensions.

| Subset | Train engines | Test engines | Operating conditions | Fault modes |
|---|---|---|---|---|
| FD001 | 100 | 100 | 1 | 1 |
| FD002 | 260 | 259 | 6 | 1 |
| FD003 | 100 | 100 | 1 | 2 |
| FD004 | 248 | 249 | 6 | 2 |

### Data dictionary

| Columns | Meaning |
|---|---|
| unit | Engine identifier; use for grouping, not as a numeric predictor |
| cycle | Operational cycle index |
| setting_1 to setting_3 | Operating conditions/settings |
| sensor_1 to sensor_21 | Sensor measurements |

A sensor can be nearly constant or redundant. That is a reason to investigate it, not an automatic instruction to drop it. Keep a record of each removed feature and the evidence for removal.

## 5.2 Phase A — Data and feature pipeline

### Step A1: Ingest and validate the raw files

Create explicit column names and validate the shape. Some whitespace-delimited files have trailing blank columns when read, so remove only columns that are entirely empty and then check the expected 26 columns.

```
import pandas as pd

COLUMNS = (
    ["unit", "cycle", "setting_1", "setting_2", "setting_3"]
    + [f"sensor_{i}" for i in range(1, 22)]
)

def load_cmapss(path):
    data = pd.read_csv(path, sep=r"\s+", header=None)
    data = data.dropna(axis=1, how="all")
```

```
    if data.shape[1] != len(COLUMNS):
        raise ValueError(
            f"Expected {len(COLUMNS)} columns, got {data.shape[1]}"
        )

    data.columns = COLUMNS
    return data.sort_values(["unit", "cycle"]).reset_index(drop=True)
```

Run checks after loading:

- Every engine has increasing cycle numbers.
- One row represents one engine-cycle snapshot.
- All expected sensors are numeric.
- The unique engine count matches the dataset documentation.
- Missing values, infinite values, duplicate engine-cycle keys, and unexpected columns are reported explicitly.

Do not silently clean a failed assertion. Log the condition and decide whether it is a data issue or an expected property.

### Step A2: Construct the RUL target correctly

For a training engine that ends at cycle `T`, the raw target at cycle `t` is:

$$\mathrm{RUL}(i, t) = T_i - t$$

A common modeling choice is the capped target:

$$\mathrm{RUL_cap}(i, t) = \min(\mathrm{RUL}(i, t), c), \text{ with } c = 125$$

The cap means that very early engine life is treated as the same broad “healthy” range rather than asking the model to distinguish, for example, 210 remaining cycles from 220 based on weak sensor evidence. **The cap is a modeling choice**, not an unquestionable truth; later, test uncapped and capped targets as a controlled experiment.

```
last_cycle = train.groupby("unit")["cycle"].max()
train["rul_raw"] = train["unit"].map(last_cycle) - train["cycle"]
train["rul_capped"] = train["rul_raw"].clip(upper=125)
```

Only run this target construction on the run-to-failure training trajectories. The test target file has a different role: it gives the true RUL at each test engine's final observed cycle for benchmark evaluation.

### Step A3: Perform focused exploratory analysis

Build these plots before feature engineering:

19. One engine trajectory for several sensors, with cycle on the x-axis.
20. Engine lifetime distribution.
21. Sensor variance and near-constant-sensor table.
22. Sensor correlation heatmap.
23. For FD002, distributions of operating settings and selected sensors by operating regime.

Questions to answer in the report:

- Which sensors change as failure approaches?
- Which measurements appear flat or redundant?
- Do operating settings explain shifts that could be mistaken for degradation?
- Are trends similar across engines or highly engine-specific?

Do not make causal claims from correlation alone.

### Step A4: Build strictly trailing-window features

A row-level model treats each cycle as a feature vector. Rolling features give the model a compact history of each sensor. For a window of the most recent `w` cycles, create features such as:

- rolling mean,
- rolling standard deviation,
- rolling minimum and maximum,

- rolling slope (whether the sensor is rising or falling).

Start with windows of 20, 30, and 50 cycles as experiments. A rolling feature at cycle `t` must use only observations at or before `t`. **Do not use a centered rolling window**, because it includes future measurements.

For each engine, sort by cycle before calculating rolling features. Pick one consistent policy for incomplete early windows: either drop rows until a full window is available, or use a documented `min_periods` policy and ensure the same policy is used at inference. Never calculate rolling features across engine boundaries.

### Step A5: Treat normalization as a design choice

For FD001, begin with unnormalized tree-based models and a training-fitted scaler for linear models. For FD002, investigate operating-regime effects. You can include the operating settings as predictors, compare condition-aware normalization, or cluster operating settings using training data only and normalize measurements relative to those regimes.

Avoid normalizing every engine using statistics computed over its **entire** trajectory when you claim to predict RUL at earlier cycles. Those statistics can include future sensor readings relative to the prediction time. Prefer transformations based only on the permitted history or statistics fitted on the training fold. Document exactly how normalization behaves for a previously unseen engine.

### Step A6: Select features based on evidence

Use a staged approach:

24. Remove only invalid or constant features confirmed on the training fold.
25. Build a simple feature set with a few rolling statistics.
26. Compare validation performance with and without sensor groups.
27. Optionally evaluate mutual information, correlation with target, L1 regularization, or model-based importance.
28. Check whether feature selection is stable across engine-level folds.

Do not select features after inspecting the final test outcomes.

## 5.3 Phase B — Split, baseline, and model comparison

### Step B1: Split by engine ID, not by row

Consecutive rows from one engine are strongly related. If rows from the same engine appear in both training and validation, validation can look artificially easy. Split by `unit` so every engine belongs to only one partition.

```
from sklearn.model_selection import GroupShuffleSplit

# `frame` contains engineered examples; `unit` is retained for grouping.
# Never include `unit` itself in X as a numeric model feature.
X = frame.drop(columns=["unit", "rul_raw", "rul_capped"])
y = frame["rul_capped"]
groups = frame["unit"]

splitter = GroupShuffleSplit(
    n_splits=1, test_size=0.20, random_state=42
)
train_idx, valid_idx = next(splitter.split(X, y, groups=groups))

X_train, X_valid = X.iloc[train_idx], X.iloc[valid_idx]
y_train, y_valid = y.iloc[train_idx], y.iloc[valid_idx]
```

The final submitted benchmark test set is separate from the training trajectories. Use a group-based validation split (or GroupKFold) within the training trajectories for model decisions. Do not tune against the benchmark test labels.

### Step B2: Build baselines first

Use at least these baselines:

- **Constant baseline:** predict the median training RUL.

- **Ridge regression:** tests whether a simple linear model can use the window features.
- **Random forest or ExtraTrees:** a non-linear baseline with modest setup effort.
- **Gradient boosting:** HistGradientBoosting in scikit-learn, or XGBoost/LightGBM as an optional comparison.

Record scores in a table. A complex model that does not beat a justified baseline is not a successful result merely because it is more advanced.

### Step B3: Put preprocessing and model steps in one pipeline

A pipeline should include the scaler/selector when required and the estimator. Fit all learned preprocessing on `X_train` only. This avoids accidentally scaling or selecting features based on validation values.

### Step B4: Tune only after validation works

Tune in this order:

29. window length,
30. capped versus uncapped target,
31. selected sensor/feature groups,
32. model family and regularization,
33. estimator-specific parameters.

Use randomized search for a small search, and Optuna for a larger, budgeted search. Log every trial and use the same split policy across compared models. Do not expand the search simply to find a nicer score.

## 5.4 Evaluation: use metrics that match maintenance risk

### RMSE and MAE

$$\text{MAE} = (1/n)\ \Sigma\ |\hat{y}_i - y_i|$$

$$\text{RMSE} = \text{sqrt}((1/n)\ \Sigma\ (\hat{y}_i - y_i)\text{^}2)$$

MAE is the typical absolute error. RMSE gives larger errors greater influence.

### NASA asymmetric score

For the conventional PHM08-style score, define `e = predicted_RUL - true_RUL`. An overestimate (`e > 0`) means the engine is predicted to last longer than it really does, which can delay maintenance. The per-engine penalty is commonly written as:

$$e = \text{predicted RUL} - \text{true RUL};\ \text{if } e < 0:\ s(e) = \exp(-e/13) - 1;\ \text{if } e \geq 0:\ s(e) = \exp(e/10) - 1;\ \text{total } S = \Sigma\ s(e_i).$$

Lower is better. Implement the sign convention deliberately and unit-test it: for equal absolute errors, the late-warning/overprediction side should receive the larger penalty. Check the exact convention against the benchmark reference you cite before publishing results.

### Evaluate at the correct level

For the standard C-MAPSS test evaluation, generate a prediction at the **last observed row of each test engine** and compare it with the corresponding supplied RUL truth value. Do not present metrics from every intermediate test row as though each had a supplied benchmark truth.

If the model is trained with capped labels, report both:

- scores against raw test RUL (the operational quantity), and
- scores against `min(true_RUL, 125)` (the capped-target comparison).

Label the two clearly. Include a separate uncapped-target experiment if you want to test whether the cap helps.

### Required plots

- predicted versus true RUL,
- residual versus true RUL,
- error distribution,
- per-engine table of worst overestimates and underestimates,
- score comparison by model and feature window.

Investigate whether the model becomes dangerously optimistic near failure. Average error can conceal this failure mode.

## 5.5 Interpretation and deployment

### Interpretation

Use permutation importance or SHAP only after a valid model has been selected. Explain important rolling features in domain language—for example, “the recent slope of sensor X is associated with lower predicted RUL”—rather than claiming a sensor causes failure. Check whether conclusions hold across validation folds.

### Save the full inference system

Save the trained preprocessing-plus-model pipeline, feature names, window size, sensor order, RUL cap, dataset subset, and training configuration. Saving just the estimator is insufficient if inference also requires rolling features or a scaler.

### FastAPI contract

A simple inference API can accept a JSON payload containing recent cycles, each with the three settings and 21 sensor values, and return:

```
{
  "predicted_rul_cycles": 24.7,
  "model_version": "rul-fd001-v1",
  "window_size": 30
}
```

Validate that there are enough chronological observations, values are numeric and finite, the required columns are present, and the feature-generation code matches training. Return a clear client error for invalid inputs; do not silently reorder sensor fields without validating names.

### Monitoring plan

In a real fleet, you may not immediately know actual RUL. Monitor sensor distributions, operating-setting distributions, missing/invalid input rates, prediction distribution, and—when maintenance/failure outcomes eventually arrive—delayed model error. Distribution change is a reason to investigate, not proof by itself that the model has failed.

# 6. Project B — TOC-UCO Ordinal Classification

## 6.1 Problem and ordinal structure

In ordinary multiclass classification, classes are different categories with no required order. In ordinal classification, the categories have a meaningful order. For instance, predicting class 4 instead of class 3 is generally less severe than predicting class 1 instead of class 5.

For K ordered classes:

$$y \in \{1, 2, \ldots, K\}$$

A conventional classifier can still be a useful baseline, but its standard loss treats label errors as category mismatches rather than explicitly using their distance. The central experiment is to compare a nominal classifier against an ordinal-aware method under metrics that respect the order.

## 6.2 Choose and document the dataset

The official TOC-UCO repository provides a catalogue, preprocessing details, and 30 randomized train/test partitions for reproducible comparisons. [3][4]

Recommended options:

- **Main project:** `abalone` from official TOC-UCO. The current catalogue lists 10 input variables and 9 ordered classes.
- **Second dataset for stability:** `bank1`, which the current catalogue lists with 8 input variables and 9 ordered classes.
- **Five-class alternative:** `winequalityRed`, listed in TOC-UCO with five classes.

Always inspect the downloaded files and metadata before training. Record the exact dataset key, class order, split/seed identifiers, row count, feature count, and class frequencies. Never assume a number of classes based only on an old note or a filename.

## 6.3 Phase A — Data and preprocessing

### Step A1: Define the target order

Write the ordered class mapping explicitly in the README. Map labels to consecutive integer indices only if needed by the model, and retain a reverse mapping for readable outputs. Do not sort string labels alphabetically and assume that is the correct order.

### Step A2: Honor the official partitions

Use the repository's supplied partition files for benchmark comparability. For a defensible workflow:

34. Choose the first official partition for initial pipeline development.
35. Tune models using only the training side of that partition, using cross-validation suitable for the training data.
36. Freeze the feature pipeline, metrics, and tuning rule before looking at final test results.
37. Once the method is fixed, evaluate across the official partitions to report mean and standard deviation, not only the best seed.

If you use a custom split for an additional experiment, explain why and retain the official-partition result separately.

### Step A3: Audit before cleaning

Measure missingness, duplicates, data types, target order, class frequencies, and possible invalid values. Do not automatically apply KNN imputation or MICE simply because a dataset is small. If no values are missing, there is nothing to impute. If values are missing, compare a justified simple imputer with a more complex alternative on validation data; fit it on the training fold only.

For outliers, first inspect whether they are data errors or valid extremes. Do not drop observations just because they are far from the median; deleting rows can worsen class imbalance and bias the task.

### Step A4: Preprocessing by model family

- **Linear and logistic/ordinal regression:** numeric scaling is often useful. Fit `StandardScaler` using training-fold data only.
- **Tree-based classifiers:** scaling is usually not needed, though imputation and categorical encoding still must be correct.
- **Nominal categorical features:** use a one-hot encoder unless the feature has a genuine order.
- **Ordered categorical features:** use an explicit order mapping only when domain meaning supports it.

Use `ColumnTransformer` and `Pipeline` when there are different numeric and categorical columns. Do not create target-derived input features. A feature such as "is the target above class 3?" is leakage because it uses the answer you are trying to predict.

### Step A5: Perform focused EDA

Create:

- class-frequency bar chart,
- feature distributions split by class,
- selected feature-versus-class plots to inspect monotonic patterns,
- correlation heatmap for numeric variables,
- optional PCA/UMAP plot as an exploratory visualization.

A two-dimensional PCA or UMAP plot is not proof that a model can or cannot classify the data; it is only a view of some structure after dimensionality reduction.

## 6.4 Phase B — Nominal versus ordinal models

Compare these model families.

### Family A: Nominal baseline

Start with a majority-class baseline, then a scikit-learn multiclass model such as LogisticRegression, RandomForestClassifier, ExtraTreesClassifier, or HistGradientBoostingClassifier. This gives you a fair reference for asking whether ordinal structure helps.

### Family B: Ordinal-aware model

Use an ordinal regression model, such as a cumulative-threshold / proportional-odds model or an established ordinal library supported by your environment. Libraries such as `mord` and `skordinal` are possible candidates, but verify installation, API, and maintenance against your current Python/scikit-learn versions before committing to one.

The threshold idea is straightforward: instead of treating `1, 2, ..., K` as unrelated categories, learn `K - 1` ordered questions such as "Is the label above class 1?", "above class 2?", and so on. A well-formed ordinal model should respect the ordering of these threshold probabilities.

If independently trained threshold classifiers produce inconsistent/crossing probabilities, do not blindly subtract them to obtain class probabilities; enforce or repair monotonicity using a documented method, or use an ordinal implementation that handles this structure correctly.

### Family C: Cost-sensitive ordinal experiment

As an optional advanced extension, make distant errors more costly than adjacent errors. A simple illustrative cost is:

$$C(i,j)=(i-j)^2$$

This cost matrix is not universally correct. Use it only when the task supports that assumption and explain why a two-class error should cost more than a one-class error. Instance weighting can alter the problem and should be treated as a separate experiment, not a free improvement.

### Step B1: Train a credible baseline

Record training and validation metrics for:

38. majority class,
39. nominal logistic regression,
40. a nominal tree ensemble,
41. an ordinal regression model.

Start simple enough that you can explain every transformation and loss. Do not start with a neural ordinal model unless these comparisons are already reliable.

### Step B2: Tune without test-set feedback

Tune preprocessing, regularization, tree parameters, or ordinal model settings on validation data only. Prefer a small, principled search. If the repository offers 30 partitions, do not repeatedly inspect them, tweak settings toward the best test score, and then call the result an unbiased benchmark. Freeze your model-selection procedure first, then use the partitions for stability reporting.

## 6.5 Metrics that respect order

### Mean absolute error of class index

For ordered integer class indices:

$$\text{Ordinal MAE} = (1/n)\ \Sigma\ |\hat{y}_i - y_i|$$

An error of two classes contributes twice as much as an error of one class. This makes the distance explicit.

### Off-by-one accuracy

$$\text{Off-by-one accuracy} = (1/n)\ \Sigma\ 1(|\hat{y}_i - y_i| \leq 1)$$

This reports the percentage of predictions that are correct or within one adjacent class. Define it in the README because “accuracy” alone usually means exact-match accuracy.

### Quadratic Weighted Kappa (QWK)

QWK measures agreement while penalizing disagreement more strongly as class distance increases. One common definition is:

$$\text{QWK} = 1 - [\Sigma_{ij}\ W_{ij}\ O_{ij}]\ /\ [\Sigma_{ij}\ W_{ij}\ E_{ij}], \text{ where } W_{ij} = (i-j)^2\ /\ (K-1)^2$$

Here, $O$ is the observed confusion matrix and $E$ is the expected matrix based on the true and predicted class marginals. Higher QWK is better; 1 represents perfect agreement. Use a library implementation and test it on a small known example instead of implementing it for the first time inside your main training script.

### Report all of these

- exact-match accuracy (secondary),
- ordinal MAE,
- QWK,
- off-by-one accuracy,
- confusion matrix in class order,
- performance per class where sample size allows,

- mean and standard deviation across official partitions after the method is frozen.

A model can have acceptable accuracy but make distant mistakes on minority classes. Read the confusion matrix and inspect which direction errors occur.

## 6.6 Probability output, interpretation, and deployment

For `K` classes, an ordinal API should ideally return both a hard class and probabilities. It may also return cumulative probabilities such as `P(Y <= k)` for every threshold. These values help downstream users understand uncertainty; they are not automatically calibrated just because they are probabilities.

Example response shape:

```
{
  "predicted_class": 6,
  "class_probabilities": [0.02, 0.03, 0.07, 0.12, 0.24, 0.28, 0.14, 0.07, 0.03],
  "cumulative_probabilities": [0.02, 0.05, 0.12, 0.24, 0.48, 0.76, 0.90, 0.97],
  "model_version": "ordinal-abalone-v1"
}
```

This is only a schema example. The probabilities must come from your fitted model, the number of entries must match the dataset's classes, and the cumulative vector must be non-decreasing for `P(Y <= k)`.

Use permutation importance or SHAP for interpretation. For ordinal models, inspect whether features matter differently near different thresholds if the model supports threshold-specific analysis. Avoid claims that a single global importance score proves causality.

Save the full preprocessing-plus-model pipeline and the class mapping. Test that:

- predicted classes are within the defined class range,
- class probabilities are finite, non-negative, and sum to approximately 1,
- cumulative probabilities are ordered and within `[0, 1]`,
- the returned label matches the stored reverse mapping,
- the API rejects invalid or missing required inputs.

## 7. Reproducibility and experiment tracking

Use a single results table with one row per experiment. At minimum, store:

| Field | Example |
|---|---|
| project | rul_fd001 or ordinal_abalone |
| dataset_version | Source URL, archive hash, or repository commit |
| split_id | group_split_seed_42 or official partition ID |
| seed | 42 |
| features | rolling_mean_std_slope_w30 |
| target_config | cap_125 or class-order key |
| model | Ridge, ExtraTrees, LogisticAT |
| params | Serialized parameters/configuration |
| metrics | RMSE/MAE/NASA score or ordinal MAE/QWK |
| artifact | Saved pipeline identifier |

### Decision log template

For each consequential choice, record:

- **Decision:** what did you select?
- **Reason:** which data property or requirement motivated it?
- **Alternatives:** what did you test?
- **Evidence:** which validation result, plot, or diagnostic supports it?
- **Trade-off:** what could be worse because of this choice?
- **Reversal condition:** what evidence would make you change it?

### Common failure modes to explicitly test

| Failure mode | How to detect it | Remedy |
|---|---|---|
| Same engine in train and validation | Compare unique engine IDs in both splits | Group split by engine |
| Preprocessing fitted on all data | Inspect pipeline and fit locations | Fit preprocessing inside each training fold |
| Future readings in a window | Audit rolling-window code; no centered windows | Use trailing windows only |
| Target column included in features | Print and assert feature names | Explicit feature whitelist/target drop |
| Wrong test target alignment | Check one truth value per test engine | Align by documented engine order/ID |
| Ordinal labels sorted alphabetically | Print mapping table | Define class order explicitly |
| Tuning against test partitions | Review experiment log and decision timing | Freeze model-selection rule before final runs |
| Reporting only one convenient metric | Inspect report tables | Report all preselected metrics and failure cases |
| API preprocessing differs from training | Contract test saved pipeline against training examples | Reuse the same transformer code and schema |

## 8. Six-week build roadmap

This is a milestone plan, not a race. Do not advance until the exit criteria for each week are satisfied.

### Week 1 — C-MAPSS ingestion and target

- Download and version the data.
- Write a parser with schema assertions.
- Audit engine counts and cycle sequences.
- Construct raw and capped RUL targets.
- Produce the first EDA report.

**Exit criteria:** another run reproduces the same tables and plots; the RUL labels pass hand-checked examples.

### Week 2 — C-MAPSS feature engineering and baseline

- Build strictly trailing rolling features.
- Create an engine-level validation split.
- Train constant and Ridge baselines.
- Add a tree-based baseline.
- Report RMSE, MAE, and asymmetric score.

**Exit criteria:** code proves there is no engine overlap between train and validation, and at least one non-constant model is evaluated fairly.

### Week 3 — C-MAPSS improvements and inference

- Compare window sizes and capped versus uncapped targets.

- Test feature reduction or a boosted tree.
- Analyze overestimates near failure.
- Serialize the complete pipeline and build a basic API.
- Optional: move to FD002 with an explicit operating-condition plan.

**Exit criteria:** one command can reproduce metrics and a valid API request returns a RUL estimate.

### Week 4 — Ordinal data and nominal baselines

- Download the official TOC-UCO package.
- Confirm the dataset key, class count, label order, and official partition format.
- Build class-balance and feature-distribution plots.
- Train majority-class, logistic, and tree baselines.
- Report ordinal MAE, QWK, off-by-one accuracy, and confusion matrix.

**Exit criteria:** the target ordering and split IDs are documented; baseline metrics can be reproduced.

### Week 5 — Ordinal-aware model and stability

- Add an ordinal regression or cumulative-threshold method.
- Check probability consistency and model assumptions.
- Compare against nominal baselines under the same partitions.
- Freeze the model-selection rule and evaluate across official partitions.
- Analyze distant and class-specific errors.

**Exit criteria:** you can explain whether ordinal learning helped, where it did not, and how stable the result is.

### Week 6 — Packaging and portfolio polish

- Build and test the ordinal API.
- Save both inference pipelines and configuration files.
- Add tests for schemas, metrics, and prediction contracts.
- Complete both READMEs and decision logs.
- Capture clean plots and a short demo of each project.

**Exit criteria:** each project has setup instructions, one-command evaluation, documented limitations, saved model artifacts or reproducible training steps, and a working inference demonstration.

## 9. Portfolio README and interview narrative

### Recommended README structure for each project

42. **Problem and use case** — what is predicted, by whom, and why it matters.
43. **Dataset provenance** — source, citation, version/hash, license note, schema.
44. **Evaluation design** — split policy and why it matches the problem.
45. **Methodology** — target, features, preprocessing, baselines, model candidates.
46. **Results** — the complete comparison table, not only the best row.
47. **Error analysis** — where and how the model fails.
48. **Deployment** — input contract, output contract, local run command, tests.
49. **Decision log** — choices and rejected alternatives.
50. **Limitations** — simulated data, distribution shift, class imbalance, calibration, or data constraints.
51. **Reproduction** — environment setup, commands, seeds, and expected outputs.

### Interview story for the pair

> *"I built two supervised-learning systems focused on evaluation correctness. In C-MAPSS, I prevented leakage by validating on held-out engines and used only trailing history to construct features. I compared capped and*

> *uncapped RUL targets and reported symmetric regression metrics alongside the asymmetric maintenance score. In TOC-UCO, I compared a nominal multiclass baseline with an ordinal-aware method using ordinal MAE and quadratic weighted kappa. In both projects, I documented failure cases, preserved the complete inference pipeline, and made the results reproducible."*

Only use this narrative after the actual experiments support every statement. Replace general claims with your measured results and honest caveats.

### What not to write

Avoid statements such as "state-of-the-art," "industry-ready," or "highly accurate" without a fair benchmark, a numerical result, and a meaningful baseline. Do not claim deployment or drift monitoring if you only built a notebook. A small, tested API is a stronger claim than a production claim you cannot substantiate.

## 10. Definition of done

### C-MAPSS RUL project

- ☐ Dataset source, subset, and version recorded.
- ☐ Schema and engine/cycle checks pass.
- ☐ Raw and capped target definitions explained.
- ☐ Rolling features use only trailing history and never cross engine boundaries.
- ☐ Train/validation split has disjoint engine IDs.
- ☐ Baseline and stronger model are compared under the same split.
- ☐ Final test predictions use the last observed row of each test engine.
- ☐ RMSE, MAE, and the asymmetric score are reported with the sign convention documented.
- ☐ Errors near failure and worst per-engine cases are reviewed.
- ☐ Saved inference pipeline includes feature logic and metadata.
- ☐ API schema validation and tests pass.
- ☐ README includes decisions, limitations, and reproduction steps.

### TOC-UCO ordinal project

- ☐ Official dataset key, class count, label order, and partition IDs recorded.
- ☐ Missing values, data types, duplicate records, and class balance audited.
- ☐ Preprocessing is fitted only on training folds.
- ☐ A nominal baseline is compared against an ordinal-aware model.
- ☐ Ordinal MAE, QWK, off-by-one accuracy, and confusion matrix are reported.
- ☐ Official partitions are used for reproducibility and stability, without tuning to test scores.
- ☐ Probability outputs pass range, sum, and ordering checks.
- ☐ Saved pipeline includes preprocessing and class mapping.
- ☐ API tests pass.
- ☐ README includes assumptions, limitations, and reproduction steps.

### Portfolio overall

- ☐ Each project has a clean Git history and no raw-data secrets or unnecessary large files.
- ☐ Results are reproducible from documented commands.
- ☐ Metrics table includes all meaningful baselines and model candidates.
- ☐ Every key modeling choice is supported by evidence.
- ☐ Project claims can be defended line by line in an interview.

## 11. References and official resources

**[1] NASA Open Data Portal — C-MAPSS Jet Engine Simulated Data**

https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data

Dataset description, 26-column format, four subsets, and benchmark objective.

**[2] Zenodo — PCoE Turbofan Engine Degradation Simulation**

https://zenodo.org/records/15346912

Current record listing for `CMAPSSData.zip` (check the record again before citing a version in a publication).

**[3] TOC-UCO GitHub repository**

https://github.com/ayrna/tocuco

Official catalogue, dataset characteristics, class counts, and benchmark resources.

**[4] TOC-UCO paper (arXiv)**

https://arxiv.org/abs/2507.17348

Research paper describing the tabular ordinal classification repository and its 46-dataset / 30-partition benchmark design.

**Useful implementation references**

- scikit-learn model-selection and pipeline documentation: https://scikit-learn.org/stable/modules/cross_validation.html and https://scikit-learn.org/stable/modules/compose.html
- scikit-learn `GroupKFold`: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html
- scikit-learn Cohen's kappa: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.cohen_kappa_score.html
- FastAPI tutorial: https://fastapi.tiangolo.com/tutorial/
- Optuna documentation: https://optuna.readthedocs.io/
- SHAP documentation: https://shap.readthedocs.io/

## Final principle

The strongest part of these projects is not the fanciest algorithm. It is the chain of reasoning from the data contract to the split, from the target and loss to the metric, from the error analysis to the deployment contract. If you can defend that chain and reproduce it, these projects become credible evidence of practical ML engineering.